LABORATORIO 01

CONSOLIDACIÓN Y VERIFICACIÓN DE ESQUEMAS LÓGICOS FUNCIONALES

OBJETIVO

Organizar y validar las entidades del proyecto de Grados y Títulos en esquemas especializados, aislando responsabilidades y controlando el acceso por roles (RBAC).

FUNDAMENTO TEÓRICO

Agrupar en esquemas lógicos evita usar todo en el esquema por defecto (dbo) y permite asignar permisos por módulo de negocio, manteniendo ordenadas las bases de datos grandes.

PROCEDIMIENTO

1. Elegir la BD: USE GradosTitulos

2. Verificar con IF NOT EXISTS

3. Crear cada esquema (CREATE SCHEMA)

4. Auditar en sys.schemas

PUNTOS CLAVE

- Esquemas: Catalogo (datos maestros), Academico (personas y estudios), Tramite (trámites y pagos), Evaluacion (evaluaciones), Documento (documentos), Seguridad (roles y accesos).
- RolMesaDePartes: SELECT en Catalogo; SELECT/INSERT/UPDATE en Tramite; DENY en Seguridad.
- RolComiteEvaluador: SELECT en Academico; SELECT/INSERT/UPDATE en Evaluacion; DENY en Seguridad.
- RolAuditorSeguridad: SELECT en Seguridad.

CÓDIGO SQL

```
USE GradosTitulos;
GO
IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name='Catalogo')
    EXEC('CREATE SCHEMA Catalogo;');
-- idem: Academico, Tramite, Evaluacion, Documento, Seguridad
GO
SELECT schema_id, name AS NombreEsquema FROM sys.schemas
WHERE name IN ('Catalogo','Academico','Tramite',
               'Evaluacion','Documento','Seguridad');
```

PREGUNTA DE VALIDACIÓN

¿De qué manera la separación de entidades en esquemas lógicos mejora la aplicación del principio de mínimo privilegio en el control de acceso corporativo (RBAC)?

Idea clave:

Los permisos se otorgan por esquema: cada rol accede solo a los módulos que necesita, en lugar de recibir permisos amplios sobre todo dbo.

LABORATORIO 01 - EVIDENCIA DE EJECUCIÓN

CÓDIGO SQL EJECUTADO

SQLQuery3.sql - localhost.GradosTitulos (sa (67))*

```sql
USE GradosTitulos;
GO
-- Creación y validación de esquemas lógicos corporativos
IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'Catalogo')
     EXEC('CREATE SCHEMA Catalogo;');

IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'Academico')
     EXEC('CREATE SCHEMA Academico;');

IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'Tramite')
     EXEC('CREATE SCHEMA Tramite;');

IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'Evaluacion')
     EXEC('CREATE SCHEMA Evaluacion;');

IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'Documento')
     EXEC('CREATE SCHEMA Documento;');

-- Creación de roles de base de datos
IF NOT EXISTS (SELECT * FROM sys.database_principals WHERE name = 'RolMesaDePartes')
     CREATE ROLE RolMesaDePartes;

IF NOT EXISTS (SELECT * FROM sys.database_principals WHERE name = 'RolComiteEvaluador')
     CREATE ROLE RolComiteEvaluador;

IF NOT EXISTS (SELECT * FROM sys.database_principals WHERE name = 'RolAuditorSeguridad')
     CREATE ROLE RolAuditorSeguridad;

-- Conceder y denegar permisos sobre esquemas
GRANT SELECT ON SCHEMA::Catalogo TO RolMesaDePartes;
GRANT SELECT, INSERT, UPDATE ON SCHEMA::Evaluacion TO RolMesaDePartes;

GRANT SELECT ON SCHEMA::Academico TO RolComiteEvaluador;
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::Evaluacion TO RolComiteEvaluador;

DENY SELECT, INSERT, UPDATE, DELETE ON SCHEMA::Seguridad TO RolAuditorSeguridad;
GRANT SELECT, INSERT, UPDATE ON SCHEMA::Seguridad TO RolAuditorSeguridad;

-- Consultar los permisos de los roles
SELECT
     dp.name  AS RolResponsable,
     perm.state_desc AS Accion,
     perm.permission_name AS Permiso,
     s.name AS EsquemaRequerido
FROM sys.database_permissions perm
INNER JOIN sys.database_principals dp ON perm.grantee_principal_id = dp.principal_id
INNER JOIN sys.schemas s ON perm.major_id = s.schema_id
WHERE perm.class_desc = 'SCHEMA'
ORDER BY dp.name, s.name;
```

100 % No se encontraron problemas.

RESULTADO: PERMISOS POR ROL Y ESQUEMA

Resultados | Mensajes

	RolCorporativo	Accion	Permiso	EsquemaProtegido
1	RolAuditorSeguridad	GRANT	SELECT	Seguridad
2	RolComiteEvaluador	GRANT	SELECT	Academico
3	RolComiteEvaluador	GRANT	INSERT	Evaluacion
4	RolComiteEvaluador	GRANT	SELECT	Evaluacion
5	RolComiteEvaluador	GRANT	UPDATE	Evaluacion
6	RolComiteEvaluador	DENY	DELETE	Seguridad
7	RolComiteEvaluador	DENY	INSERT	Seguridad
8	RolComiteEvaluador	DENY	SELECT	Seguridad
9	RolComiteEvaluador	DENY	UPDATE	Seguridad
10	RolMesaDePartes	GRANT	SELECT	Catalogo
11	RolMesaDePartes	DENY	DELETE	Seguridad
12	RolMesaDePartes	DENY	INSERT	Seguridad
13	RolMesaDePartes	DENY	SELECT	Seguridad
14	RolMesaDePartes	DENY	UPDATE	Seguridad
15	RolMesaDePartes	GRANT	INSERT	Tramite
16	RolMesaDePartes	GRANT	SELECT	Tramite
17	RolMesaDePartes	GRANT	UPDATE	Tramite

LABORATORIO 02

IMPLEMENTACIÓN DE ÍNDICES CLÚSTER ESTRATÉGICOS EN TABLAS PRINCIPALES

OBJETIVO

Diseñar y verificar los índices clúster en las llaves primarias de las tablas clave del proyecto, para lograr un ordenamiento físico óptimo de los registros en disco.

FUNDAMENTO TEÓRICO

El índice clúster determina el orden físico en que se guardan las filas. Una clave adecuada (como un ID autoincremental secuencial) previene la fragmentación excesiva de páginas en inserciones masivas.

PROCEDIMIENTO

1. Consultar sys.indexes

2. Unir con sys.tables y sys.schemas

3. Filtrar i.type = 1 (CLUSTERED)

4. Ordenar por esquema y tabla

PUNTOS CLAVE

- Solo existe un índice clúster por tabla: su nivel hoja contiene las filas reales.
- Si se intenta crear un segundo índice clúster, SQL Server devuelve el error 1902.
- Para otros órdenes (p. ej. por DNI) se usan índices NONCLUSTERED.

CÓDIGO SQL

```
USE GradosTitulos;
GO
SELECT s.name AS Esquema, t.name AS Tabla,
       i.name AS IndiceClustered, i.type_desc AS Tipo
FROM sys.indexes i
INNER JOIN sys.tables t ON i.object_id = t.object_id
INNER JOIN sys.schemas s ON t.schema_id = s.schema_id
WHERE i.type = 1  -- 1 indica CLUSTERED
ORDER BY s.name, t.name;
```

PREGUNTA DE VALIDACIÓN

Explique por qué una tabla de base de datos relacional en SQL Server solo puede tener un único índice clúster.

Idea clave:
Porque el índice clúster ES el orden físico de las filas, y los datos solo pueden estar ordenados físicamente de una única manera.

LABORATORIO 02 - EVIDENCIA DE EJECUCIÓN

CÓDIGO SQL EJECUTADO

SQLQuery4.sql - localhost.GradosTitulos (sa (67))*

```sql
USE GradosTitulos;
GO
-- ============================================================
-- PASO 1: Verificar que la tabla Academico.Persona ya tiene 1 índice Clúster
-- (Creado en su clave primaria IdPersona)
-- ============================================================
SELECT
    s.name AS Esquema,
    t.name AS Tabla,
    i.name AS NombreIndice,
    i.type_desc AS TipoIndice
FROM sys.indexes i
INNER JOIN sys.tables t ON i.object_id = t.object_id
INNER JOIN sys.schemas s ON t.schema_id = s.schema_id
WHERE s.name = 'Academico'
    AND t.name = 'Persona'
    AND i.type = 1; -- 1 = CLUSTERED
GO
-- ============================================================
-- PASO 2: Demostración del límite físico (Intentar crear un 2do índice Clúster)
-- SQL Server bloqueará la operación porque las tablas ya están ordenadas por 1
-- ============================================================
BEGIN TRY
    CREATE CLUSTERED INDEX IX_Persona_SegundoClustered
     ON Academico.Persona (DNI);
END TRY
BEGIN CATCH
    SELECT
        ERROR_NUMBER() AS NumeroError,
        ERROR_MESSAGE() AS MensajeDelMotorSQL;
END CATCH;
GO

-- ============================================================
-- PASO 3: Solución correcta cuando ya existe un índice Clúster
-- Para ordenar por una segunda columna (DNI), se usa un índice NONCLUSTERED
-- ============================================================
IF NOT EXISTS (
    SELECT 1 FROM sys.indexes
    WHERE name = 'IX_Persona_DNI_Demo'
        AND object_id = OBJECT_ID('Academico.Persona')
)
BEGIN
    CREATE NONCLUSTERED INDEX IX_Persona_DNI_Demo
        ON Academico.Persona (DNI);
END
GO
```

RESULTADO: ÍNDICE CLÚSTER EXISTENTE

Resultados | Mensajes

	Esquema	Tabla	NombreIndice	TipoIndice
1	Academico	Persona	PK_Persona	CLUSTERED

RESULTADO: ERROR 1902 AL CREAR UN 2.º CLÚSTER

	NumeroError	MensajeDelMotorSQL
1	1902	Cannot create more than one clustered index on t...

LABORATORIO 03

CREACIÓN DE ÍNDICES NO CLÚSTER CON COLUMNAS INCLUIDAS

OBJETIVO

Diseñar índices no clúster con la cláusula INCLUDE para evitar búsquedas costosas de tipo Key Lookup en consultas frecuentes de padrones y reportes.

FUNDAMENTO TEÓRICO

Los índices no clúster guardan las columnas clave y punteros a los datos. Con INCLUDE se añaden columnas no clave en las hojas del índice, logrando consultas totalmente cubiertas (covering indexes).

PROCEDIMIENTO

1. Verificar que el índice no exista

2. CREATE NONCLUSTERED sobre Persona (DNI)

3. INCLUDE de las columnas a leer

4. Repetir para TipoPersona

PUNTOS CLAVE

- Sin INCLUDE: Index Seek + Key Lookup por cada fila (más lecturas y más costo).
- Con INCLUDE: Index Seek directo; todas las columnas salen del propio índice.
- El mensaje 'Los comandos se han completado correctamente' es normal: CREATE INDEX es DDL, no una consulta de lectura.

CÓDIGO SQL

```
IF NOT EXISTS (SELECT * FROM sys.indexes
    WHERE name='IX_Persona_DNI_Cover'
    AND object_id=OBJECT_ID('Academico.Persona'))
BEGIN
    CREATE NONCLUSTERED INDEX IX_Persona_DNI_Cover
    ON Academico.Persona (DNI)
    INCLUDE (Nombres, ApellidoPaterno, ApellidoMaterno,
             Correo, TipoPersona);
END;
-- 2.o: IX_Persona_TipoPersona ON (TipoPersona)
--      INCLUDE (DNI, Nombres, ApellidoPaterno, ApellidoMaterno)
```

PREGUNTA DE VALIDACIÓN

¿Qué diferencia de rendimiento existe entre una consulta que utiliza un índice cubierto con INCLUDE frente a otra que requiere un operador de Key Lookup?

Idea clave:

Con INCLUDE todo se lee del propio índice; con Key Lookup se hace un salto extra a la tabla por cada fila, lo que implica más lecturas (I/O) y más tiempo.

LABORATORIO 03 - EVIDENCIA DE EJECUCIÓN

CÓDIGO SQL EJECUTADO

SQLQuery5.sql - localhost.GradosTitulos (sa (67))*

```sql
USE GradosTitulos;
GO

-- Índice No Clúster con cobertura para búsquedas por DNI
IF NOT EXISTS (
      SELECT * FROM sys.indexes
      WHERE name = 'IX_Persona_DNI_Cover'
            AND object_id = OBJECT_ID('Academico.Persona')
)
BEGIN
      CREATE NONCLUSTERED INDEX IX_Persona_DNI_Cover
      ON Academico.Persona (DNI)
      INCLUDE (Nombres, ApellidoPaterno, ApellidoMaterno, Correo, TipoPersona);
END
GO

-- Índice No Clúster para filtrado por Tipo de Persona
IF NOT EXISTS (
      SELECT * FROM sys.indexes
      WHERE name = 'IX_Persona_TipoPersona'
            AND object_id = OBJECT_ID('Academico.Persona')
)
BEGIN
      CREATE NONCLUSTERED INDEX IX_Persona_TipoPersona
      ON Academico.Persona (TipoPersona)
      INCLUDE (DNI, Nombres, ApellidoPaterno, ApellidoMaterno);
END
GO

-- Índice No Clúster para búsqueda por Tipo de Persona (ya existe un índice NoClustered)
IF NOT EXISTS (
      SELECT * FROM sys.indexes
      WHERE name = 'IX_Persona_TipoPersona'
            AND object_id = OBJECT_ID('Academico.Persona')
)
BEGIN
      CREATE NONCLUSTERED INDEX IX_Persona_TipoPersona
      ON Academico.Persona (TipoPersona)
      INCLUDE (DNI, Nombres, ApellidoPaterno, ApellidoMaterno);
END
GO
```

RESULTADO DE LA EJECUCIÓN

Mensajes

```
Los comandos se han completado correctamente.

Hora de finalización: 2026-10-22T13:51:12.5150695-05:00
```

LABORATORIO 04

INDEXACIÓN COMPUESTA PARA CONSULTAS ANALÍTICAS DE TRÁMITES

OBJETIVO

Construir índices compuestos no clúster según los patrones de filtrado por rangos de fechas y estados administrativos, para agilizar los reportes gerenciales.

FUNDAMENTO TEÓRICO

En consultas que combinan igualdad (estado del trámite) y rangos (fecha de inicio), el orden de las columnas del índice compuesto es crítico para que el optimizador lo aproveche.

PROCEDIMIENTO

1. Separar columnas de igualdad y de rango

2. Orden: igualdad primero, rango después

3. Agregar INCLUDE para cubrir

4. Crear índices de Trámite y Pago

PUNTOS CLAVE

- IX_Tramite_Estado_Fecha en Tramite.Tramite: claves (IdEstadoTramite, FechaInicio); INCLUDE (IdMatricula, IdTipoTramite, IdCoordinacion).
- IX_Pago_Tramite_Fecha en Tramite.Pago: claves (IdTramite, FechaPago); INCLUDE (Monto, NroRecibo, TipoPago).
- Analogía: una guía telefónica ordenada por Apellido y luego por Nombre.

CÓDIGO SQL

```
USE GradosTitulos;
GO
CREATE NONCLUSTERED INDEX IX_Tramite_Estado_Fecha
ON Tramite.Tramite (IdEstadoTramite, FechaInicio)
INCLUDE (IdMatricula, IdTipoTramite, IdCoordinacion);

CREATE NONCLUSTERED INDEX IX_Pago_Tramite_Fecha
ON Tramite.Pago (IdTramite, FechaPago)
INCLUDE (Monto, NroRecibo, TipoPago);
```

PREGUNTA DE VALIDACIÓN

¿Por qué el orden de las columnas en un índice compuesto (ColumnaA, ColumnaB) afecta directamente la capacidad de búsqueda del optimizador cuando el filtro solo utiliza ColumnaB?

Idea clave:

El índice está ordenado primero por ColumnaA y luego por ColumnaB dentro de cada valor de A; sin filtrar por A no se puede ir directo al rango de B.

LABORATORIO 04 - EVIDENCIA DE EJECUCIÓN

CÓDIGO SQL EJECUTADO

SQLQuery6.sql - localhost.GradosTitulos (sa (67))*

```sql
USE GradosTitulos;
GO

-- Índice compuesto para reportes analíticos de trámites por Estado y Rango de Fecha
IF NOT EXISTS (
    SELECT * FROM sys.indexes
    WHERE name = 'IX_Tramite_Estado_Fecha'
      AND object_id = OBJECT_ID('Tramite.Tramite')
)
BEGIN
    CREATE NONCLUSTERED INDEX IX_Tramite_Estado_Fecha
      ON Tramite.Tramite (IdEstadoTramite, FechaInicio)
      INCLUDE (IdMatricula, IdTipoTramite, IdCoordinacion);
END
GO

-- Índice para agilizar la consulta de pagos por trámite y fecha
IF NOT EXISTS (
    SELECT * FROM sys.indexes
    WHERE name = 'IX_Pago_Tramite_Fecha'
      AND object_id = OBJECT_ID('Tramite.Pago')
)
BEGIN
    CREATE NONCLUSTERED INDEX IX_Pago_Tramite_Fecha
      ON Tramite.Pago (IdTramite, FechaPago)
      INCLUDE (Monto, NroRecibo, TipoPago);
END
GO

```

RESULTADO DE LA EJECUCIÓN

Mensajes

```
Los comandos se han completado correctamente.

Hora de finalización: 2026-10-22T13:57:58.3524036-05:00
```

LABORATORIO 05

AUDITORÍA, MONITOREO Y ANÁLISIS DE FRAGMENTACIÓN DE ÍNDICES

OBJETIVO

Evaluar el estado operativo de los índices: detectar duplicados o sin uso y medir el nivel de fragmentación de las estructuras de indexación del proyecto.

FUNDAMENTO TEÓRICO

Con operaciones intensivas de inserción, actualización y borrado (DML), los índices sufren fragmentación interna. Monitorear su estado permite aplicar mantenimiento: reorganizar o reconstruir.

PROCEDIMIENTO

1. Inventario de índices (sys.indexes)

2. Medir la fragmentación física

3. Filtrar fragmentación mayor a 10 %

4. Decidir: REORGANIZE o REBUILD

PUNTOS CLAVE

- 0 - 5 %: estado sano, sin acción.
- 5 - 30 %: ALTER INDEX ... REORGANIZE (mantenimiento ligero).
- Más de 30 %: ALTER INDEX ... REBUILD (reconstrucción completa).
- El script lista los índices con más de 10 % de fragmentación junto con su page_count.

CÓDIGO SQL

```
SELECT s.name AS Esquema, t.name AS Tabla,
       i.name AS NombreIndice, i.type_desc AS TipoIndice,
       i.is_primary_key, i.is_unique
FROM sys.indexes i  -- JOIN sys.tables / sys.schemas
WHERE i.name IS NOT NULL;

SELECT ps.avg_fragmentation_in_percent, ps.page_count
FROM sys.dm_db_index_physical_stats
     (DB_ID(),NULL,NULL,NULL,'LIMITED') ps
WHERE ps.avg_fragmentation_in_percent > 10;
```

PREGUNTA DE VALIDACIÓN

¿Cuál es el umbral de fragmentación recomendado en SQL Server a partir del cual se debe evaluar la reconstrucción (REBUILD) frente a una simple reorganización (REORGANIZE) de un índice?

Idea clave:

Referencia habitual de Microsoft: entre 5 % y 30 % conviene REORGANIZE; por encima de 30 % se evalúa REBUILD.

LABORATORIO 05 - EVIDENCIA DE EJECUCIÓN

CÓDIGO SQL EJECUTADO

SQLQuery1.sql - local...GradosTitulos (sa (67))*

```sql
USE GradosTitulos;
GO

-- 1. Auditoria general de todos los índices creados por esquema y tabla
SELECT
    s.name AS Esquema,
    t.name AS Tabla,
    i.name AS NombreIndice,
    i.type_desc AS TipoIndice,
    i.is_primary_key AS EsClavePrimaria,
    i.is_unique AS EsUnico
FROM sys.indexes i
    INNER JOIN sys.tables t ON i.object_id = t.object_id
    INNER JOIN sys.schemas s ON t.schema_id = s.schema_id
    WHERE i.name IS NOT NULL
    ORDER BY s.name, t.name, i.type;
GO


-- 2. Análisis de fragmentación física de los índices en el sistema

SELECT
    OBJECT_SCHEMA_NAME(ps.object_id) AS Esquema,
    OBJECT_NAME(ps.object_id) AS Tabla,
    i.name AS Indice,
    ps.avg_fragmentation_in_percent AS PorcentajeFragmentacion,
    ps.page_count AS CantidadPaginas
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED') ps
    INNER JOIN sys.indexes i ON ps.object_id = i.object_id AND ps.index_id = i.index_id
WHERE ps.avg_fragmentation_in_percent > 10
ORDER BY ps.avg_fragmentation_in_percent DESC;
GO
```

RESULTADO: AUDITORÍA DE ÍNDICES

Resultados | Mensajes

	Esquema	Tabla	NombreIndice	TipoIndice	EsClavePrimaria	EsUnico
1	Academico	Docente	PK_Docente	CLUSTERED	1	1
2	Academico	Docente	UQ_Docente_Persona	NONCLUSTERED	0	1
3	Academico	Facultad	PK_Facultad	CLUSTERED	1	1
4	Academico	Facultad	UQ_Facultad_Nombre	NONCLUSTERED	0	1
5	Academico	GradoTituloCatalogo	PK_GradoTituloCatalogo	CLUSTERED	1	1
6	Academico	Matricula	PK_Matricula	CLUSTERED	1	1
7	Academico	Persona	PK_Persona	CLUSTERED	1	1
8	Academico	Persona	UQ_Persona_DNI	NONCLUSTERED	0	1
9	Academico	Persona	IX_Persona_DNI_Demo	NONCLUSTERED	0	0
10	Academico	Persona	IX_Persona_DNI_Cover	NONCLUSTERED	0	0
11	Academico	Persona	IX_Persona_TipoPersona	NONCLUSTERED	0	0
12	Academico	ProgramaEstudios	PK_ProgramaEstudios	CLUSTERED	1	1
13	Academico	ProgramaEstudios	UQ_Programa_Nombre_Modalidad	NONCLUSTERED	0	1
14	Catalogo	EstadoTramite	PK_EstadoTramite	CLUSTERED	1	1
15	Catalogo	EstadoTramite	UQ_EstadoTramite_Descripcion	NONCLUSTERED	0	1
16	Catalogo	LineaInvestigacion	PK_LineaInvestigacion	CLUSTERED	1	1
17	Catalogo	LineaInvestigacion	UQ_LineaInvestigacion_Nombre	NONCLUSTERED	0	1
18	Catalogo	TipoTramite	PK_TipoTramite	CLUSTERED	1	1
19	Catalogo	TipoTramite	UQ_TipoTramite_Descripcion	NONCLUSTERED	0	1
20	dbo	sysdiagrams	PK__sysdiagr__C2B05B61737D8...	CLUSTERED	1	1

Esquema	Tabla	Indice	PorcentajeFragmentacion	CantidadPaginas